TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Programación 1

Sistema de Control de Inventario

Informe de diseño y fundamentación

Alumno: Valentino Mangini | **Trabajo Práctico Integrador**

1. De qué trata el trabajo

Una ferretería local necesita digitalizar el control de sus productos para evitar pérdidas de stock. Este trabajo resuelve el problema con un programa en Python que funciona por consola mediante un menú interactivo de siete opciones: cargar herramientas, mostrar el inventario, consultar stock, listar agotados, dar de alta un producto nuevo, registrar ventas o ingresos de mercadería y salir. "Tiempo real" significa que cada operación actualiza el stock en el momento, mientras el programa está en ejecución; los datos viven en memoria y no se guardan al cerrar.

El informe explica cómo se pensó la solución antes de programar, qué concepto de Python se aplicó en cada parte del código y cómo fluye el programa en sus operaciones principales.

2. Lenguaje, entorno y recursos de desarrollo

Python 3.12. Es un lenguaje de programación interpretado, de sintaxis clara y de propósito general. Se usó la versión 3.12 (la misma con la que se probó el programa). Se eligió porque es el lenguaje de la materia, porque permite representar el inventario con listas y diccionarios de forma muy directa y porque trae incorporado el manejo de excepciones (try/except/raise) que el trabajo exige.

Visual Studio Code. Es un editor de código gratuito de Microsoft, con extensiones para Python que resaltan la sintaxis y permiten ejecutar el programa desde una terminal integrada. Se eligió porque es liviano, es el que pide el enunciado para mostrar el código en el video y permite usar un tema de fondo claro con letra grande.

Terminal / consola. El programa se ejecuta y se usa desde la consola con `python inventario.py`. Se eligió una interfaz de texto porque el objetivo del trabajo es la lógica del programa (estructuras de control, funciones y errores) y no el diseño gráfico.

Estructura del proyecto. Todo el programa está en un único archivo, `inventario.py`, que es el que se ejecuta para iniciarlo.

3. Diseño previo al código

Antes de escribir código se definieron tres decisiones de diseño:

Una sola estructura de datos. El inventario es una lista de diccionarios llamada `inventario`. Cada diccionario representa una herramienta y tiene exactamente las claves `'herramienta'` y `'cantidad'` (la clave `'cantidad'` guarda el stock):

```
[{'herramienta': 'Martillo', 'cantidad': 10}, {'herramienta': 'Pala', 'cantidad': 3}]
```

Sin variables globales. `inventario` se crea dentro de `main()` y viaja como parámetro a cada función. Como las listas y los diccionarios son mutables, la función recibe una referencia a la misma lista, por lo que un `append()` o un cambio de stock hecho dentro de la función se ve en `main()` sin necesidad de devolver nada.

Dos tipos de error, dos mecanismos. Una entrada que no se puede convertir a número (por ejemplo "abc" o "3.5") hace fallar a `int()` y se captura con `try/except`. En cambio, un dato que sí es válido como texto o número pero rompe una regla de negocio (nombre vacío o duplicado, stock negativo, venta mayor al stock) se rechaza con `raise ValueError` y un mensaje descriptivo. Los casos informativos (inventario vacío, herramienta inexistente, tipo de movimiento inválido) se resuelven con una condición y un mensaje, sin excepción.

4. Funciones del programa

Función	Qué hace	Opción
mostrar_menu()	Imprime las siete opciones del menú.	-
leer_entero(mensaje)	Pide un dato y lo convierte con int(); si falla, relanza ValueError con mensaje claro.	auxiliar
validar_nombre(nombre)	Aplica strip() y lanza ValueError si el nombre queda vacío.	auxiliar
buscar_herramienta(inventario, nombre)	Recorre el inventario comparando sin mayúsculas ni espacios extremos; devuelve la herramienta o None.	auxiliar
validar_nombre_nuevo(inventario, nombre)	Valida que el nombre no esté vacío ni duplicado.	auxiliar
validar_stock_inicial(stock)	Lanza ValueError si el stock es negativo (el 0 es válido).	auxiliar
validar_unidades(unidades)	Lanza ValueError si las unidades no son mayores que 0.	auxiliar
cargar_herramientas(inventario)	Carga múltiple con reintento dentro de la función.	1
mostrar_inventario(inventario)	Lista todas las herramientas y su stock.	2
consultar_stock(inventario)	Busca una herramienta por nombre y muestra su stock.	3
reporte_agotados(inventario)	Lista solo las herramientas con stock igual a cero.	4
alta_producto(inventario)	Agrega una herramienta al final; los errores salen hacia main().	5
actualizar_stock(inventario)	Registra una venta o un ingreso siguiendo el orden de pasos del enunciado.	6
main()	Crea el inventario, repite el menú y captura los errores; la opción 7 corta el bucle.	7

5. Conceptos aplicados y su vínculo con el programa

5.1 Listas

La lista `inventario` es la única estructura donde se guardan los datos. Se crea vacía en `main()` con `inventario = []`, crece con `inventario.append(...)` en las opciones 1 y 5, se recorre con `for producto in inventario` para mostrar, buscar o reportar, y se pregunta si está vacía con `len(inventario) == 0`. Al ser mutable y pasarse por parámetro, todas las funciones trabajan sobre la misma lista.

```
inventario.append({'herramienta': nombre, 'cantidad': stock})
```

5.2 Diccionarios

Cada herramienta es un diccionario con las claves `'herramienta'` (nombre) y `'cantidad'` (stock). Se accede a sus valores por clave, por ejemplo `producto['cantidad']` para leer el stock y `producto['cantidad'] -= unidades` para descontarlo en una venta. El diccionario de una herramienta nueva se arma directamente dentro de `append()`, sin crear diccionarios sueltos.

```
producto['cantidad'] -= unidades      # venta
producto['cantidad'] += unidades      # ingreso
```

5.3 Funciones

El programa está dividido en funciones, cada una con una tarea clara. Las opciones 1 a 6 tienen su función propia, y hay funciones auxiliares de validación que se reutilizan: `validar_nombre_nuevo()` se usa tanto en la opción 1 como en la 5, y `buscar_herramienta()` se usa en las opciones 3 y 6 y en el control de duplicados. Esto evita repetir código y, si hay un error en la consulta de stock, se sabe en qué función buscarlo. Los datos compartidos se pasan como parámetros (por ejemplo `def consultar_stock(inventario)`) y se usa `return` cuando una función entrega un resultado, como el nombre validado o la herramienta encontrada. Los nombres siguen la convención snake_case.

5.4 Condicionales

Los condicionales `if/elif/else` cumplen tres roles. Primero, seleccionan la opción del menú en `main()` (`if opcion == 1 ... elif opcion != 7: raise`). Segundo, resuelven los casos informativos con un mensaje claro: inventario vacío, herramienta inexistente o tipo de movimiento inválido. Tercero, comprueban las reglas de negocio antes de lanzar `raise`, como `if unidades > producto['cantidad']` en una venta. En el reporte de agotados, un condicional sobre una variable bandera (`hay_agotados`) permite distinguir "no hay herramientas cargadas" de "ninguna herramienta tiene stock cero".

```
if tipo != '1' and tipo != 'venta' and tipo != '2' and tipo != 'ingreso':
    print('Tipo de movimiento invalido: solo se acepta venta o ingreso.')
    return
```

5.5 Bucles

Se usan tres tipos de bucle. El `while opcion != 7` de `main()` hace que el menú sea persistente: se repite hasta que el usuario elige salir. El `for numero in range(cantidad_a_cargar)` de la opción 1 repite la carga una vez por cada herramienta indicada. Y los `for producto in inventario` recorren la lista para mostrar, buscar y reportar. Además, en la opción 1 hay bucles `while` de reintento (`while nombre is None` y `while stock is None`) que vuelven a pedir solo el dato inválido y conservan lo ya correcto, por ejemplo el nombre cuando falla el stock.

```
stock = None
while stock is None:
    try:
        stock = validar_stock_inicial(leer_entero(f"Stock inicial de '{nombre}': "))
    except ValueError as e:
        print(f'Error: {e}')
```

5.6 Manejo de errores: try/except y raise

try/except captura las entradas que no se pueden convertir a entero. `leer_entero()` envuelve `int(input(...))` y relanza el error con un mensaje claro. **raise** lanza `ValueError` con un mensaje descriptivo cuando se viola una regla de negocio. Lo importante es **dónde** se captura cada error:

Opción	Dónde se captura	Qué pasa
1 (carga)	Dentro de cargar_herramientas()	Informa el error y vuelve a pedir solo el dato inválido, dentro de un bucle.
3, 5 y 6	En el except de main()	El error sale de la función; main() muestra el motivo y vuelve al menú sin modificar nada.
Menú	En el except de main()	Texto no numérico (int() falla) u opción fuera de rango (raise): muestra el error y vuelve a pedir.

```
def validar_stock_inicial(stock):
    if stock < 0:
        raise ValueError('El stock inicial no puede ser negativo.')
    return stock

# en main():
try:
    ...
except ValueError as e:
    print(f'Error: {e}')
```

6. Diagramas de flujo

Los diagramas reflejan el diseño propio del programa: el menú principal, el recorrido de carga y validación de datos (opción 1) y la actualización de stock (opción 6).

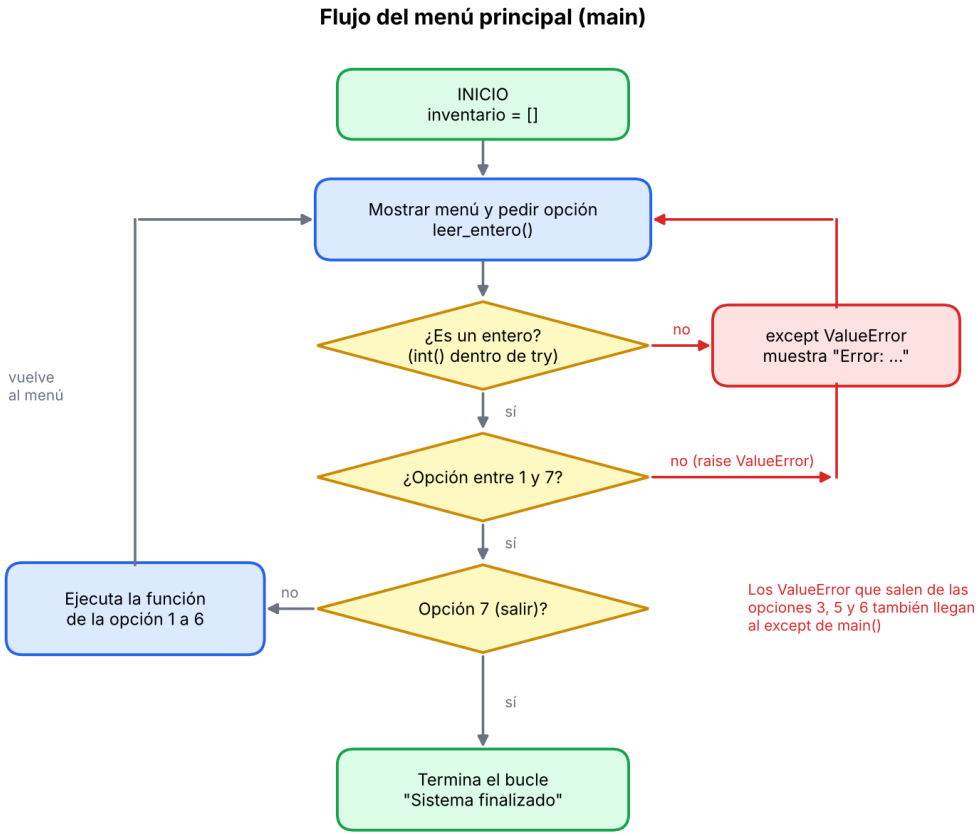


Figura 1. Flujo del menú principal.

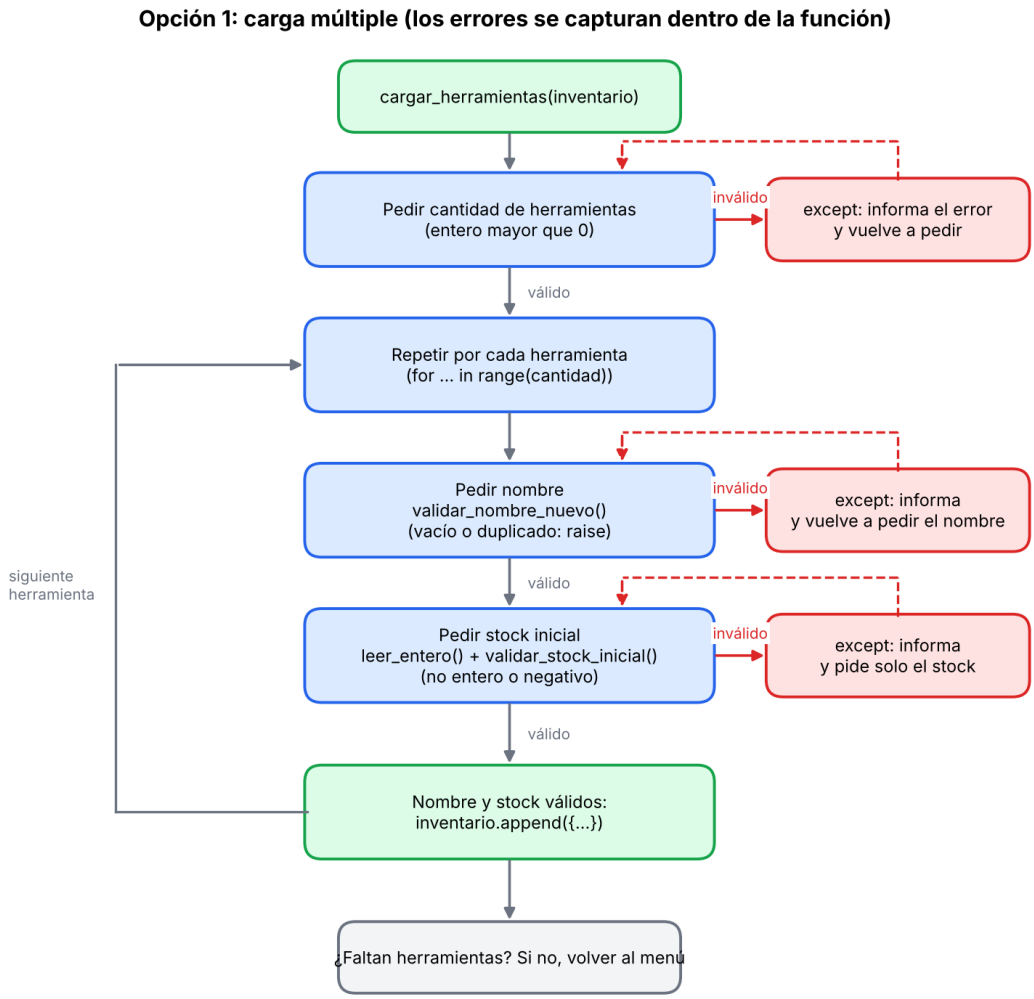


Figura 2. Carga de herramientas y validación de datos (opción 1).

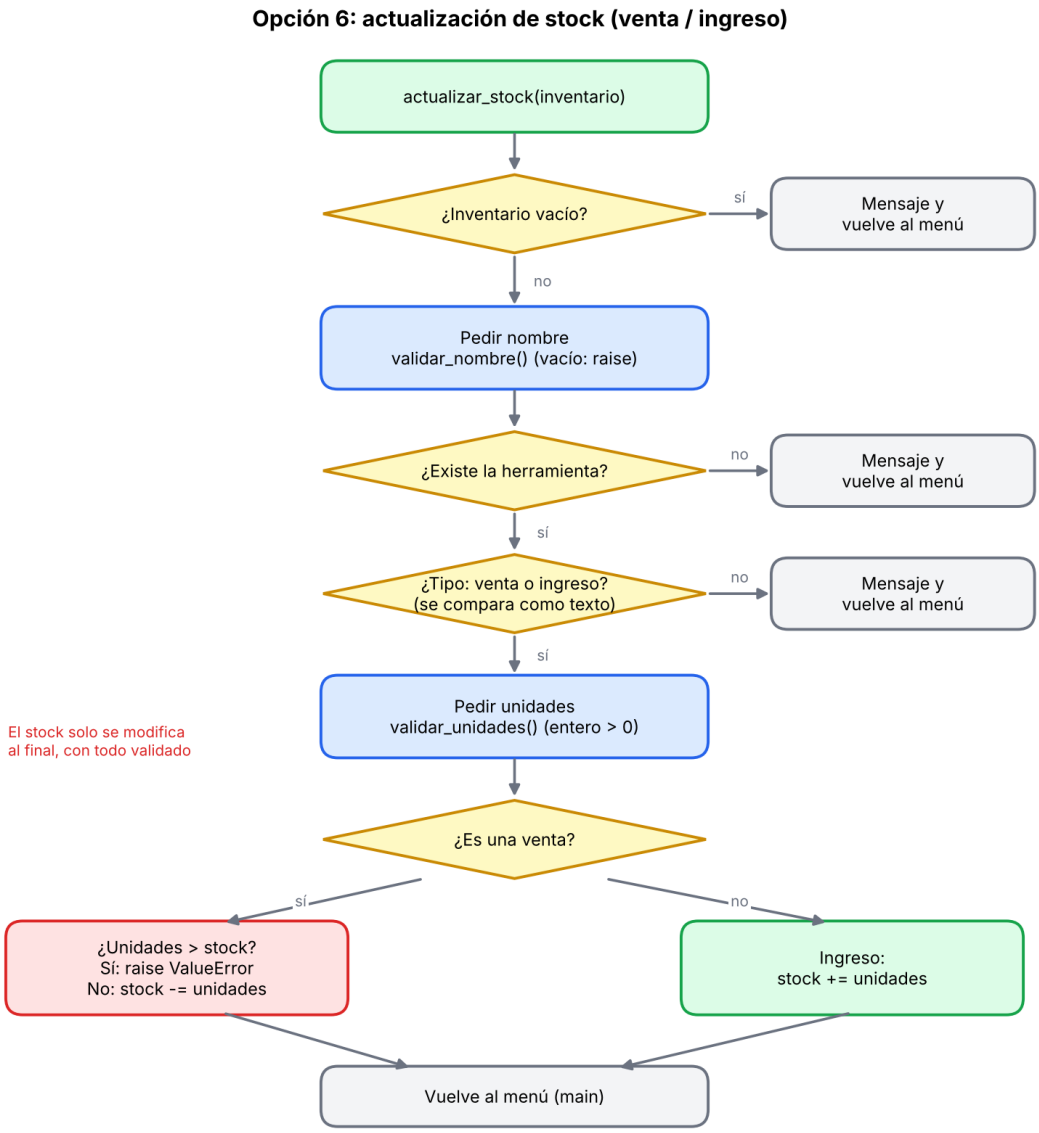


Figura 3. Actualización de stock: venta e ingreso (opción 6).

7. Validaciones implementadas

Situación	Mecanismo	Resultado
Nombre vacío o solo espacios	raise ValueError	Se rechaza con mensaje
Nombre duplicado (ignora mayúsculas y espacios)	raise ValueError	Se rechaza con mensaje
Número no entero ("abc", "3.5")	try/except (int())	Se informa el error
Stock inicial negativo (0 es válido)	raise ValueError	Se rechaza con mensaje
Cantidad de herramientas o unidades menores o iguales a 0	raise ValueError	Se rechaza con mensaje
Venta mayor al stock disponible	raise ValueError	No se modifica el stock
Opción de menú fuera de 1 a 7	raise ValueError	Se muestra el error y vuelve al menú
Herramienta inexistente o inventario vacío	condición + mensaje	No se modifica nada
Tipo de movimiento distinto de venta/ingreso	condición + mensaje	No se modifica nada

Regla general: un dato inválido nunca deja datos a medias. Una herramienta se agrega al inventario solo cuando su nombre y su stock son válidos, y un intento inválido no modifica ningún stock.

8. Pruebas realizadas

Se probó el programa con los casos de la lista de pruebas manuales del enunciado. Algunos ejemplos:

Prueba	Resultado obtenido
Opción "abc" u opción 9	Muestra el error y vuelve a pedir una opción.
Opción 1: cantidad 0, -1, abc o 3.5	Informa el error y vuelve a pedir ese dato.
Opción 1: "martillo" o " MARTILLO " con Martillo cargado	Se rechaza como duplicado.
Opción 1: Pala con stock -2, abc o 3.5	Se pide solo el stock y se conserva el nombre.
Opción 1: Pala con stock 0	Se acepta y aparece en el reporte de agotados.
Opción 3 con nombre vacío / con "Pala" inexistente	Muestra el motivo / informa que no está en el catálogo.
Opción 5: nombre vacío o duplicado	Muestra el motivo y vuelve al menú sin pedir el stock.
Opción 6: vender más que el stock	Informa el error y el stock no cambia.
Opción 6: vender 3 de 3 y luego ingresar 2	Queda en 0 (aparece como agotado) y luego en 2 (deja de aparecer).

Cómo ejecutar: abrir una terminal en la carpeta del proyecto y correr `python inventario.py`.